

SOFTWARE DEVELOPMENT PROCESSES

Lab Sheet 2 – Requirements

INTRODUCTION

This week in lectures, we talked about software requirements. Requirements form the basis of software development projects: you have to understand what you want to build before you can build it. The scenario presented in the case study is a reasonably complex system which would have a large number of requirements in the real world. In this session, you are going to try to identify the main requirements, understand how you might go about checking that a system meets them, and look at how requirements are prioritised during the software development process.

TASK 1: ELICITING REQUIREMENTS

A good starting point for the requirements process is a list of system stakeholders that have been identified. Fortunately for you, that's exactly the list you made last week! Find your notes from last week's session to remind you of the stakeholders for the scenario in the case study. For each of the identified stakeholders, discuss with the people around you how you would best go about finding out what their requirements are. Make a note of the method (or methods) you think best for each.

TASK 2: BRAINSTORMING REQUIREMENTS

Because the case study is fictional, we cannot go and interview stakeholders or observe them at work. Instead, we are going to have to rely on *brainstorming* as our primary method of identifying requirements in this session. Brainstorming isn't an ideal requirements elicitation method: what we imagine stakeholders want isn't always what they actually want. For each of the stakeholders that you have identified, talk to the people around you and make a note of a small number (i.e. 3-5) of their likely requirements. Make a note of whether each requirement is a *functional* or *non functional* requirement. Make a note of the requirements you have identified, and their type.

TASK 3: FIT CRITERIA

For each of the requirements that you identified in Task 2, try to devise a way in which you could test the system to demonstrate that the requirement is met. For most functional requirements, the fit criterion is likely to be an action that can be performed and a check to see that the expected behaviour occurs. For non functional requirements, you may need to devise a measure, metric, or experiment to offer some assurance that the requirement has been met satisfactorily. Non functional requirements are often trickier to demonstrate satisfaction of, so don't worry if you find this difficult. Make a note of the fit criteria you identified for each requirement.

TASK 4: PRIORITISATION

Although there are other methods of prioritising software requirements, by far the most commonly used in industry is the MoSCoW classification. For each of the requirements you identified, make a note as to whether you feel that the requirement should be classified as a:

- Must Have
- Should Have
- Could Have
- Won't Have

Would the stakeholder agree with your classification? How would you resolve disagreements?

TASK 5: CLASS DISCUSSION

Your lab tutor will gather everyone together with time before the end of the lab session to go through the requirements that everyone identified. See if other people came up with the same as you. Do people generally have similar requirements? Are there any obvious ones you missed? Did you come up with any that other people didn't think of? Do other people's fit criteria sound reasonable? Are people's priorities in sync?

Keep notes of the discussion and make sure you finish the session with a finalised list of requirements for the scenario in the case study. You will need them for future weeks.